# CORSO AVANZATO DI INFORMATICA E ROBOTICA

## LEZIONE 13: Python

Carlo **Zambaldo** | 04 marzo 2026

# Librerie Python e Ambienti Virtuali

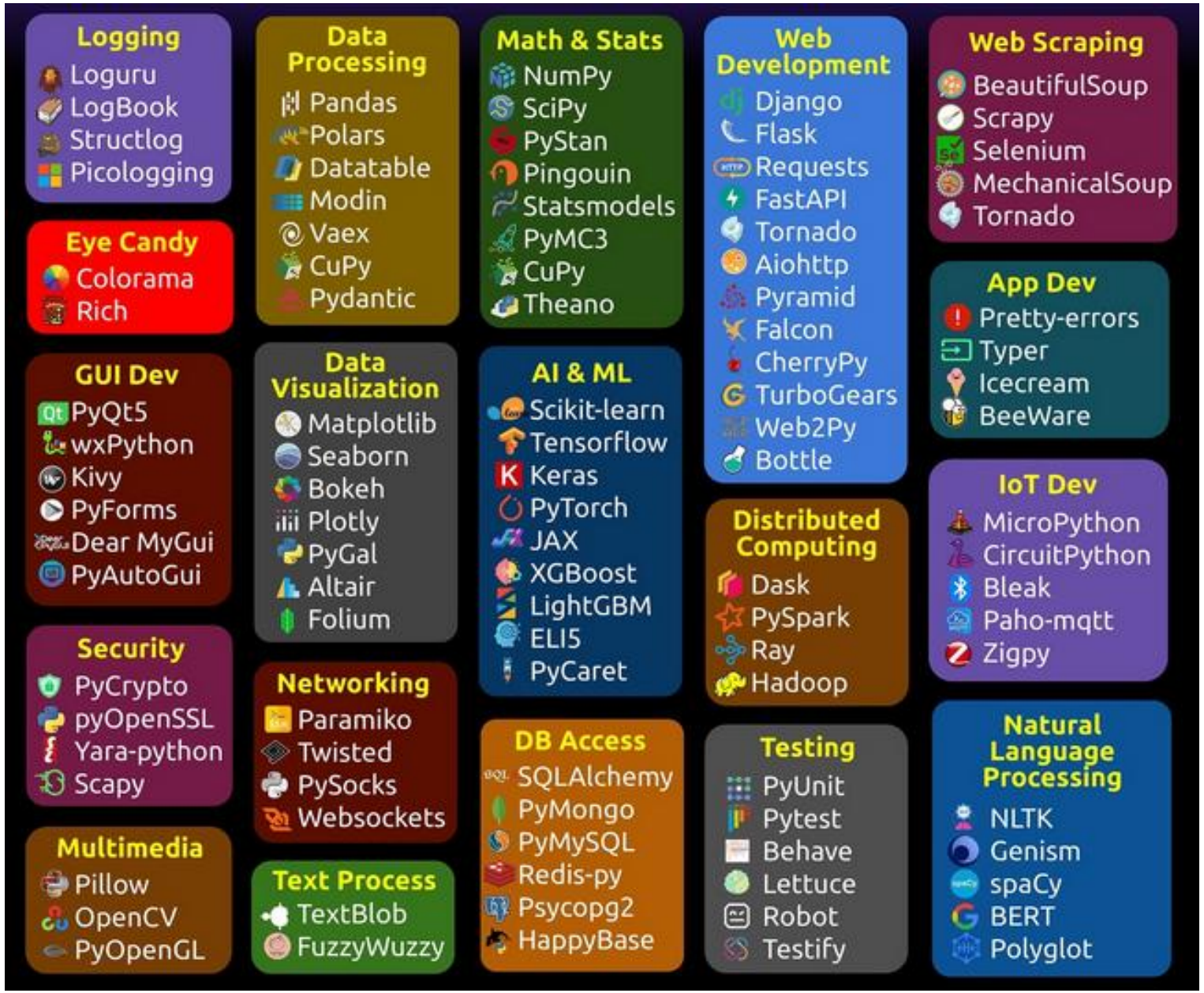
01

# Librerie Python di terze parti

Uno dei principali punti di forza del linguaggio Python è il suo vasto ecosistema di librerie.

Sebbene le librerie integrate in Python siano già molto versatili, esistono moltissime librerie e framework di terze parti che rendono la programmazione in Python ancora più produttiva.

Ecco alcune delle librerie e dei framework open-source Python più utili e popolari.



# Come importare una libreria / modulo

Arduino:

```
#include "esp_camera.h"
#include "img_converters.h"
#include "Arduino.h"
```

- **Import semplice**

```
import math
print(math.sqrt(16))
```

- **Import con alias**

```
import math as m
print(m.sqrt(25))
```

- **Importare solo una funzione**

```
from math import sqrt
print(sqrt(36))
```

| Scrittura | Quando usarla |
|---|---|
| `import modulo` | più chiaro |
| `import modulo as x` | abbreviare |
| `from modulo import funzione` | usare poche funzioni |

# “Moduli”, “pacchetti”, “librerie”: cosa stai installando davvero?

- **Modulo**: un singolo file .py (es. mio_modulo.py).
- **Pacchetto**: una cartella con codice Python (spesso con __init__.py) e magari sotto-moduli.
- **Libreria**: termine generico (spesso coincide con un pacchetto installabile).

`pip` è il **package manager ufficiale di Python**.

Un *package manager* è un programma che:

- scarica librerie
- le installa
- gestisce le versioni
- risolve le dipendenze

```
pip install <nome_pacchetto>
```

Cosa fa pip:

1. si collega a **PyPI (Python Package Index)**
2. scarica il pacchetto **`<nome_pacchetto>`**
3. scarica automaticamente le sue dipendenze
4. installa tutto dentro il tuo ambiente Python

https://pypi.org/

# Dove viene installato un modulo?

**Il problema:** su un computer possono esistere **più versioni di Python**:

- Python di sistema
- Python installato manualmente
- Python dentro un venv

Usando `pip install nome_pacchetto` non è sempre chiaro **quale Python** sta ricevendo il pacchetto, e si possono ottenere errori tipo: ModuleNotFoundError

È conveniente usare: **`python3 -m pip install nome_pacchetto`**
Questo dice al computer: «Usa pip appartenente a questo Python.»

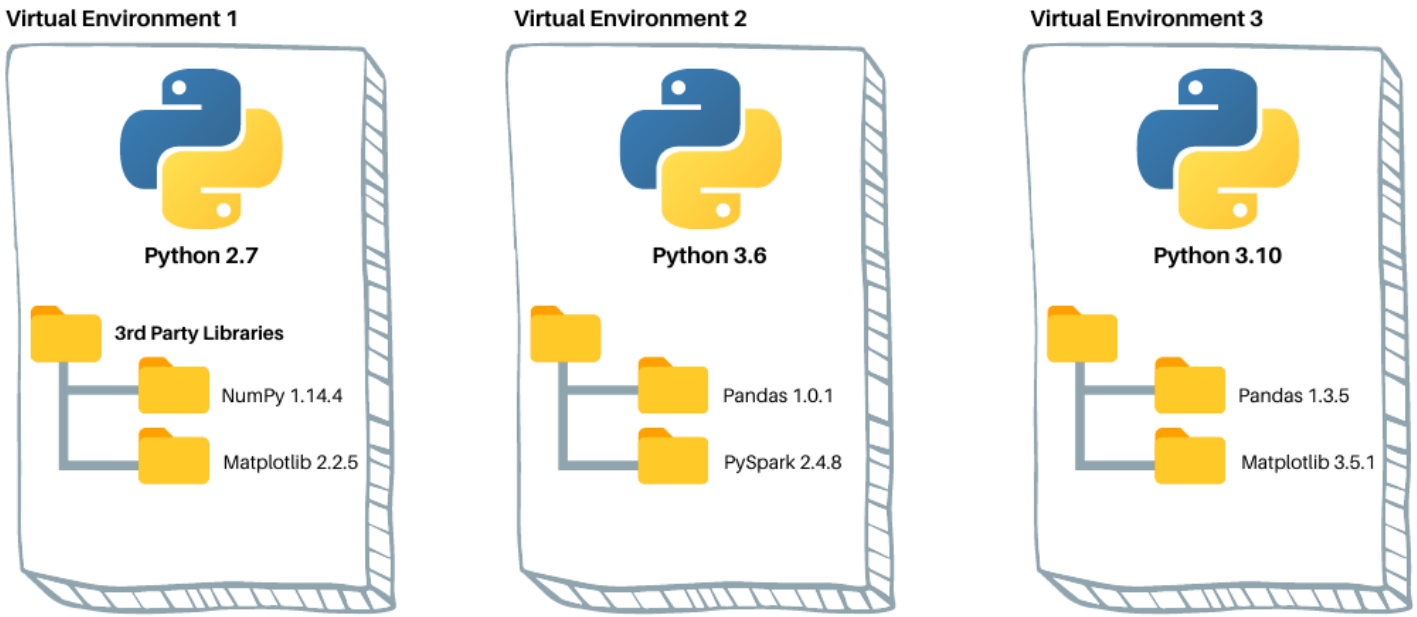
*Il problema non è installare un modulo.*
*Il problema è installarlo nel Python giusto.*

Un metodo ancora più sicuro è l'utilizzo degli ambienti virtuali

# Ambienti Virtuali
## Perché usare venv è conveniente?

Se installi tutto nel Python di sistema:

- Versioni che entrano in conflitto
- Progetti che smettono di funzionare
- Ambiente “sporco”
- Difficile replicare su un altro PC




**venv crea un ambiente Python isolato.**
**Ogni progetto ha:**

- **il suo Python**
- **le sue librerie**
- **le sue versioni**

**Con installazione globale → possibili conflitti!**

**Benefici**:

- **Nessun conflitto tra progetti**
- **Ambiente replicabile** (`pip freeze > requirements.txt` → ricrearli altrove con `pip install -r requirements.txt`)

# Accenni sui Container

| venv | Container (es. Docker / Podman) |
|---|---|
| Isola **solo Python e le sue librerie** | Isola:<br>• Python<br>• Librerie<br>• Sistema operativo<br>• Dipendenze di sistema<br>• Configurazione |
| ✔ Versioni di pacchetti indipendenti<br>✔ Leggero<br>✔ Facile da usare<br>❌ Non isola il sistema operativo<br>❌ Non isola librerie di sistema<br>Funziona **dentro il tuo computer**. | È una **mini-macchina virtuale leggera**.<br>✔ Ambiente completamente replicabile<br>✔ Funziona uguale su ogni macchina<br>✔ Ideale per server / produzione<br>❌ Più complesso<br>❌ Più pesante di un venv |

# Esempi – librerie python

02

# Esempio: salviamo il risultato di un’operazione matematica su file

```
# Operazione matematica
a = 15
b = 27
risultato = a * b

# Salvataggio su file
# 'w' sta per 'write' (sovrascrive il file se esiste già)
with open("risultato.txt", "w") as file:
    file.write(str(risultato))

print(f"Il risultato ({risultato}) è stato salvato con successo.")
```

# Esempio: salviamo il risultato di un'operazione matematica su file e se «a» e «b» venissero chiesti all'utente?

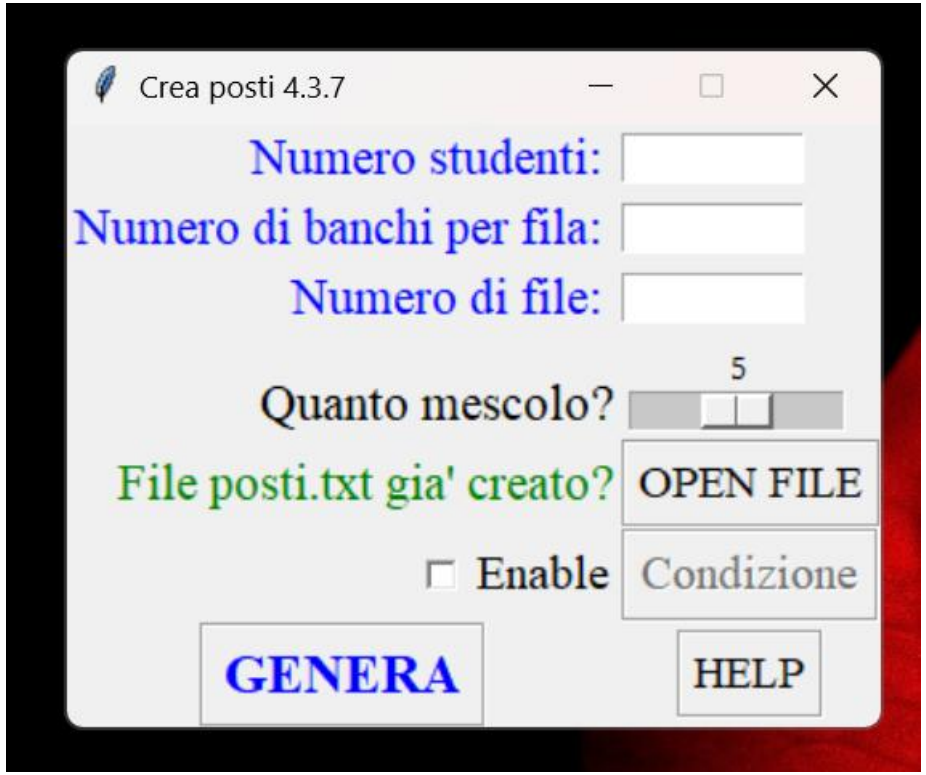
```
# Operazione matematica
a = 15          -> a = float(input("inserire a: "))
b = 27          -> b = float(input("inserire b: "))
risultato = a * b

# Salvataggio su file
# 'w' sta per 'write' (sovrascrive il file se esiste già)
with open("risultato.txt", "w") as file:
    file.write(str(risultato))

print(f"Il risultato ({risultato}) è stato salvato con successo.")
```

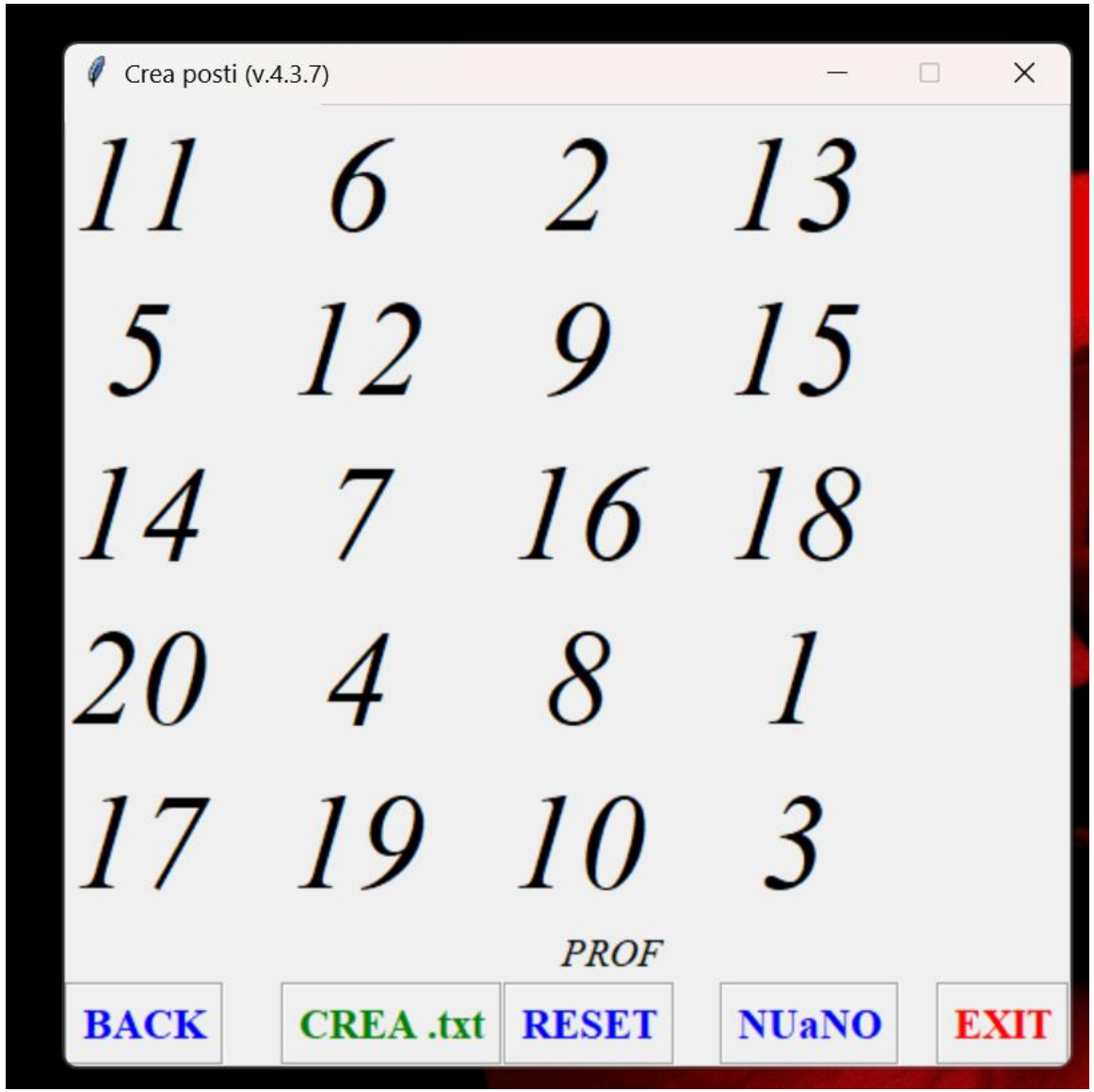
**ARDUINO:**

```
Serial.print("Il risultato (%f) è stato salvato con successo.", risultato)
```

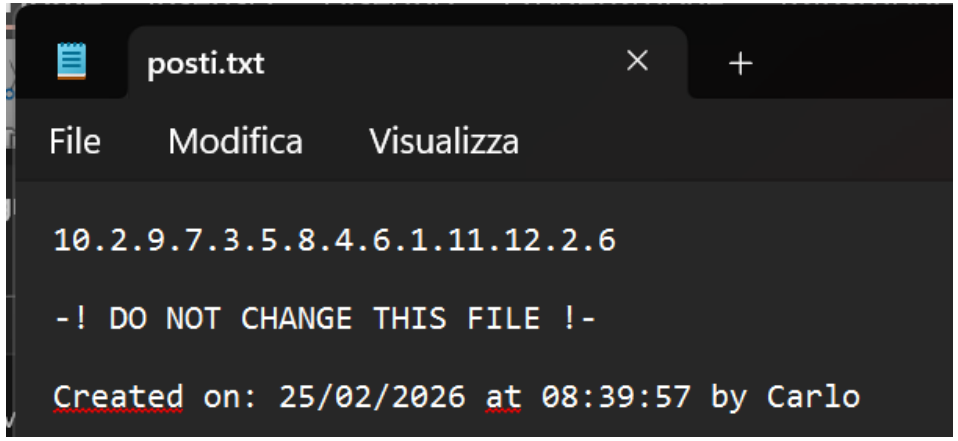
# Programma generazione posti

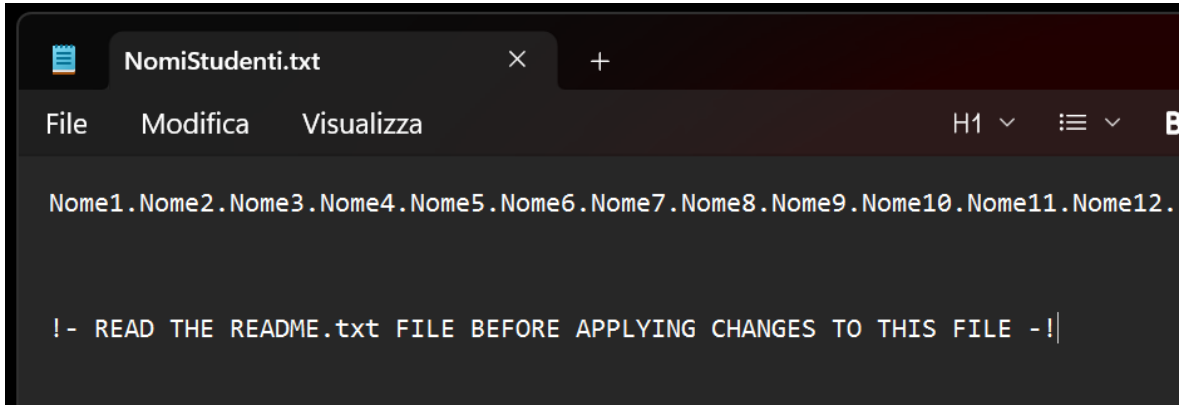
- Librerie utilizzate per il programma:
  - **time**
  - **random**
  - **tkinter**

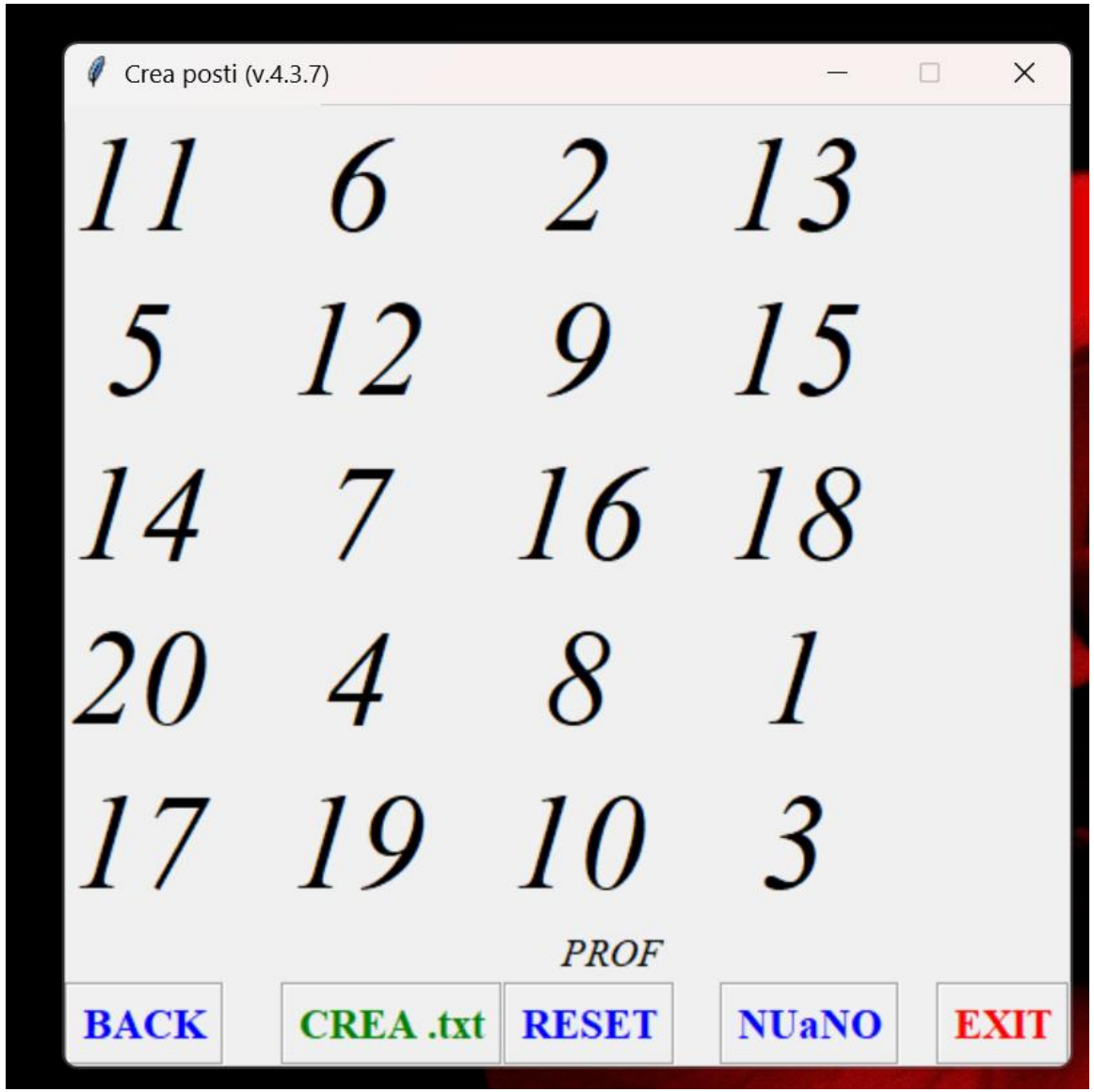


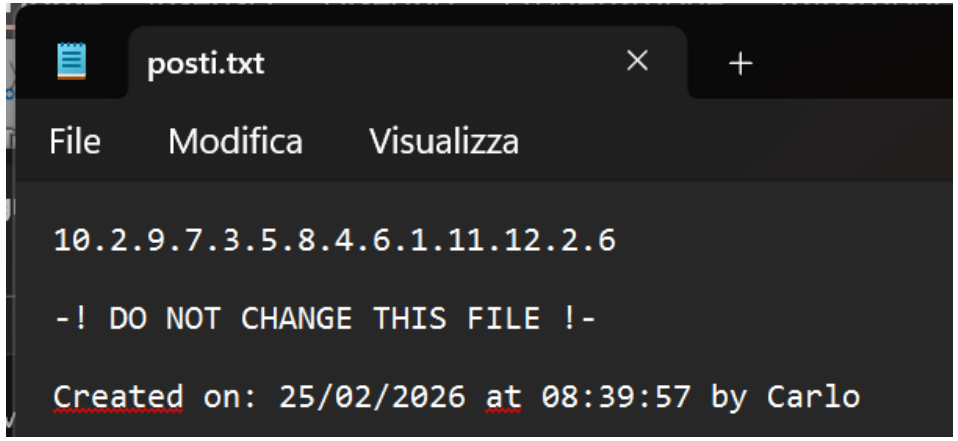
# Programma generazione posti

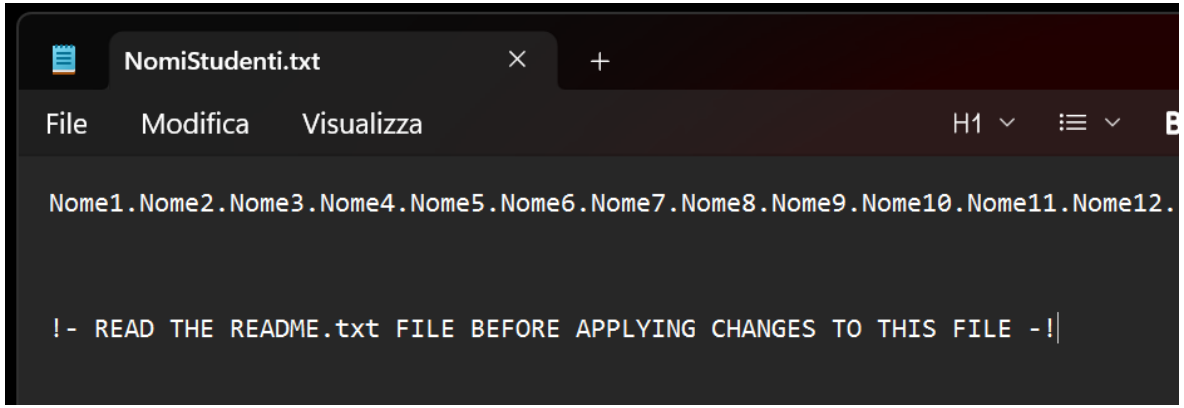


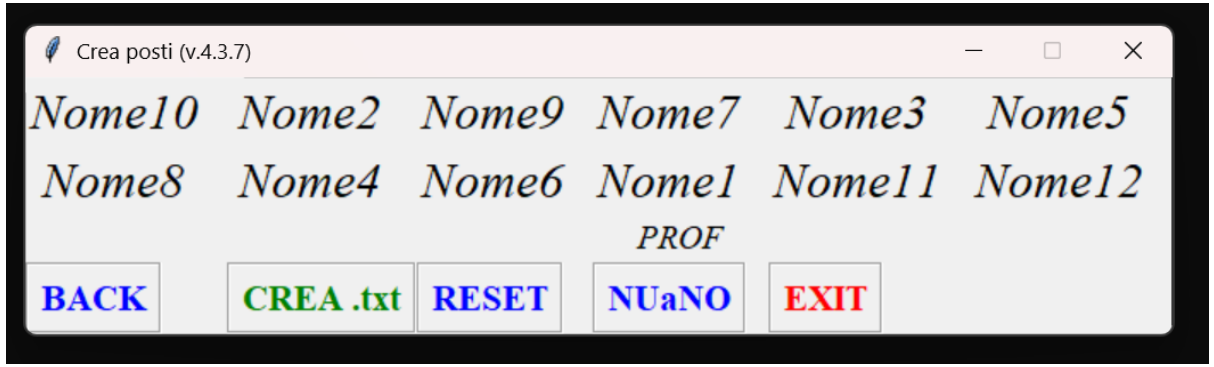



# Programma generazione posti





# GPIO Raspberry Pi 5

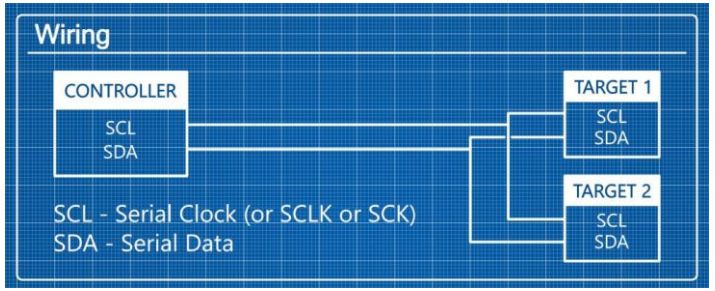
03

# GPIO

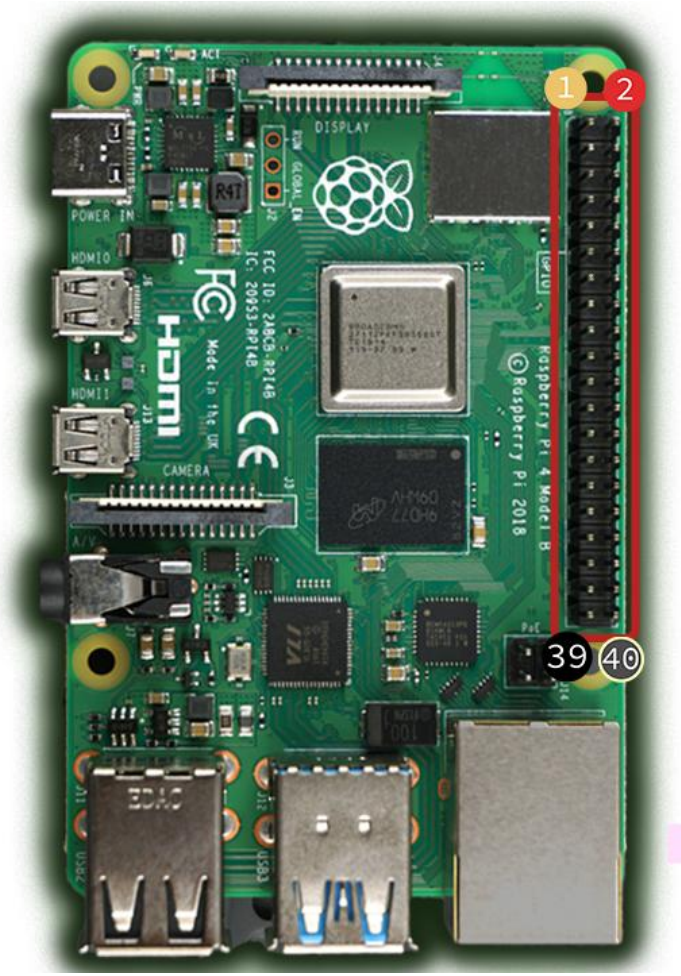
- **SPI** - Serial Peripheral Interface

https://picockpit.com/raspberry-pi/it/spi-the-serial-peripheral-interface/

- **I2C** – Inter Integrated Circuit




https://picockpit.com/raspberry-pi/i2c-the-inter-integrated-circuit-bus/

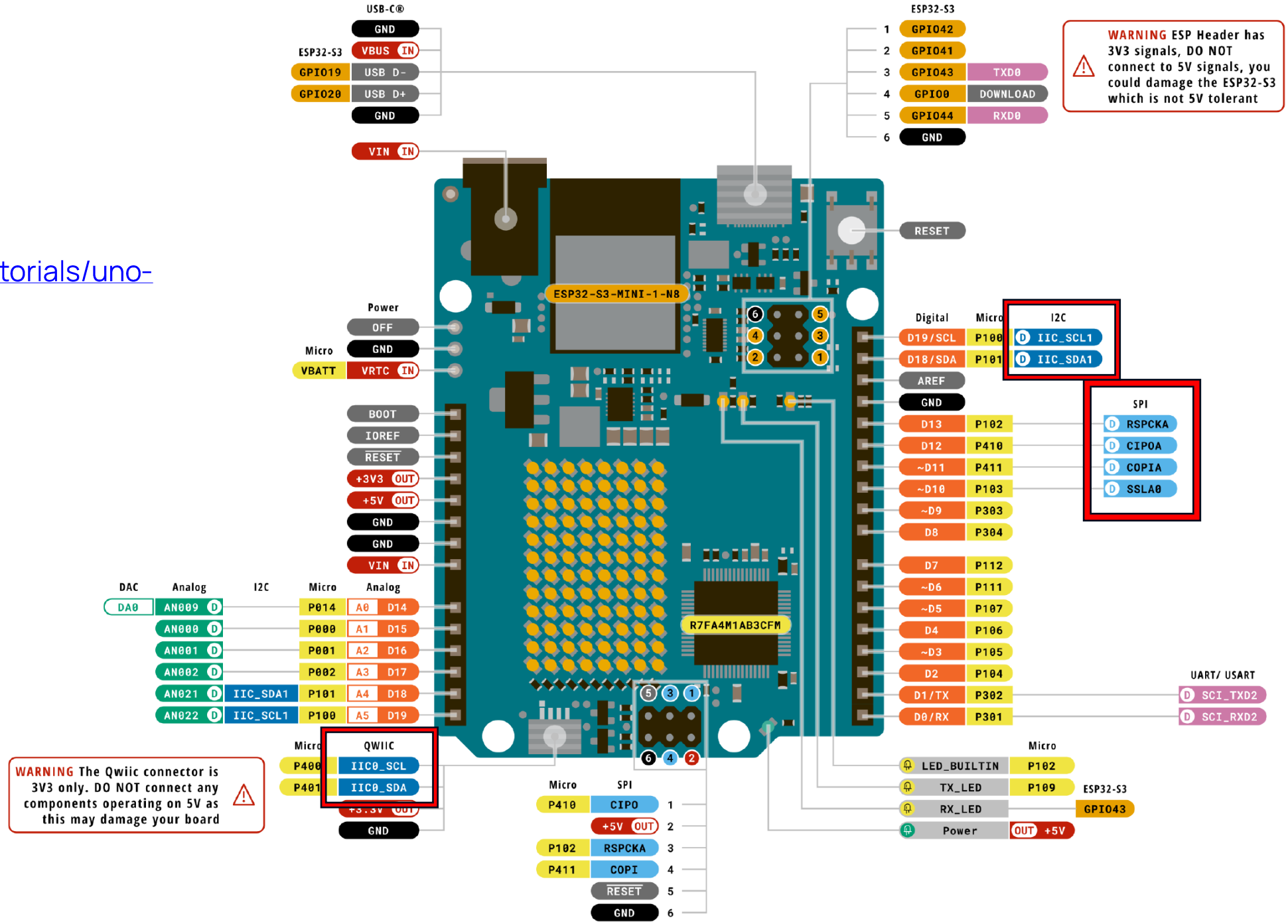



| | | | | | | |
|---|---|---|---|---|---|---|
| | | 3V3 | 1 | 2 | 5V | |
| | I2C SDA | GPIO2 | 3 | 4 | 5V | |
| | I2C SCL | GPIO3 | 5 | 6 | GND | |
| | | GPIO4 | 7 | 8 | GPIO14 | UART TX |
| | | GND | 9 | 10 | GPIO15 | UART RX |
| | | GPIO17 | 11 | 12 | GPIO18 | PCM CLK | PWM0 |
| | | GPIO27 | 13 | 14 | GND | |
| | | GPIO22 | 15 | 16 | GPIO23 | |
| | | 3V3 | 17 | 18 | GPIO24 | |
| | SPI MOSI | GPIO10 | 19 | 20 | GND | |
| | SPI MISO | GPIO9 | 21 | 22 | GPIO25 | |
| | SPI SCLK | GPIO11 | 23 | 24 | GPIO8 | SPI CE0 |
| | | GND | 25 | 26 | GPIO7 | SPI CE1 |
| | I2C ID EEPROM | GPIO0 | 27 | 28 | GPIO1 | I2C ID EEPROM |
| | | GPIO5 | 29 | 30 | GND | |
| | | GPIO6 | 31 | 32 | GPIO12 | PWM0 |
| | PWM1 | GPIO13 | 33 | 34 | GND | |
| PWM1 | PCM FS | GPIO19 | 35 | 36 | GPIO16 | |
| | | GPIO26 | 37 | 38 | GPIO20 | PCM DIN |
| | | GND | 39 | 40 | GPIO21 | PCM DOUT |

04/03/2026

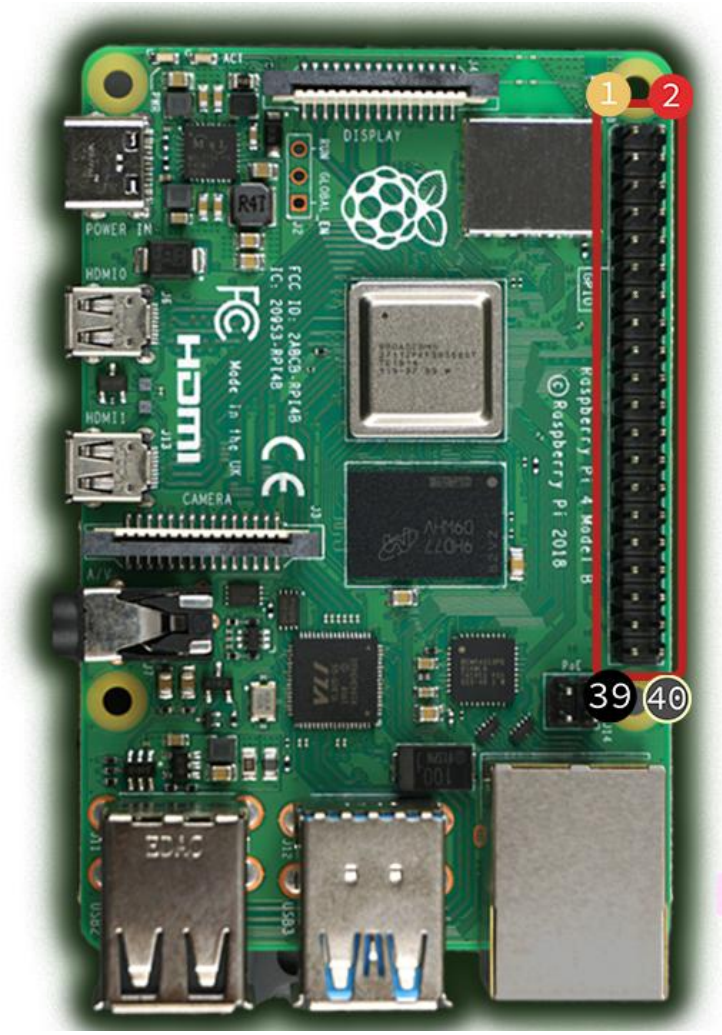
# Arduino UNO R4

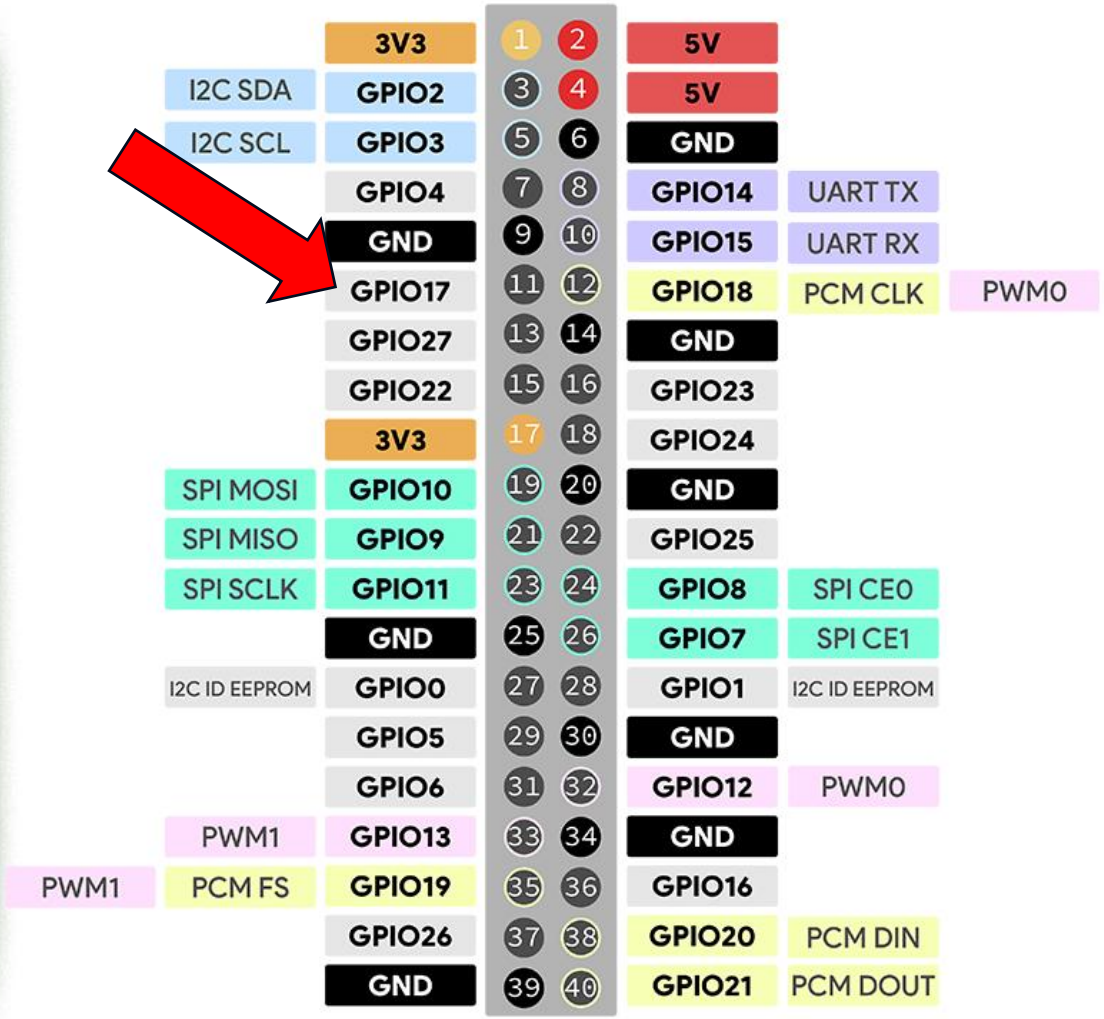
Cheat-sheet:
[https://docs.arduino.cc/tutorials/uno-r4-wifi/cheat-sheet/](https://docs.arduino.cc/tutorials/uno-r4-wifi/cheat-sheet/)



# GPIO

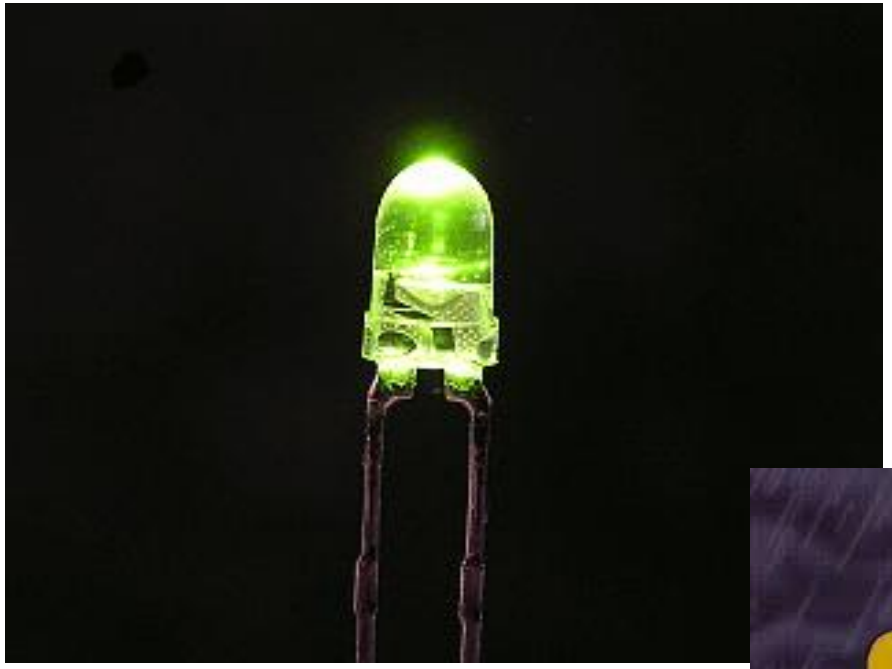
```
from gpiozero import LED
from time import sleep

# GPIO17 (BCM numbering)
led = LED(17)

# Accende il LED
led.on()
print("LED acceso")
sleep(2)

# Spegne il LED
led.off()
print("LED spento")
```



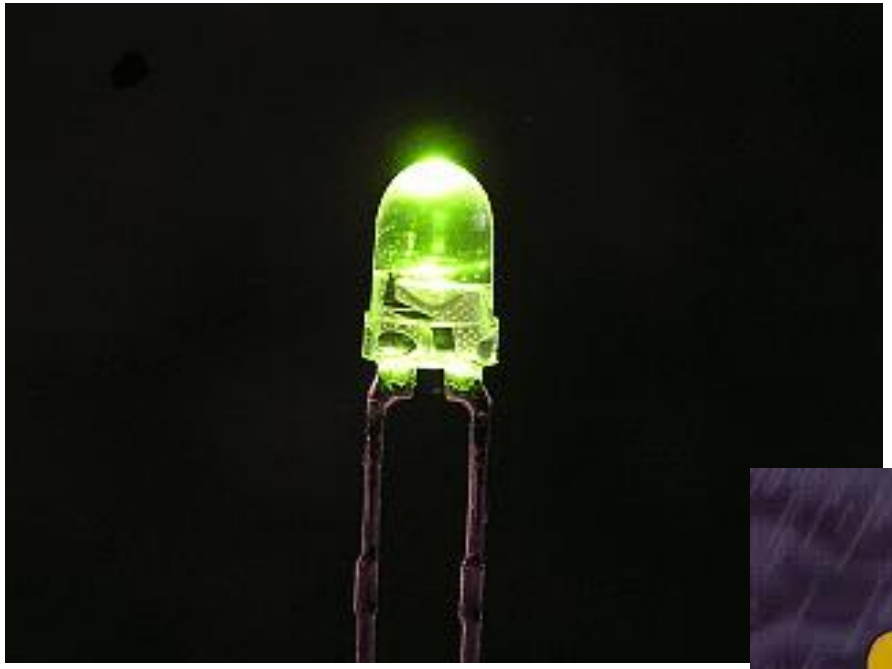
# LED blinking

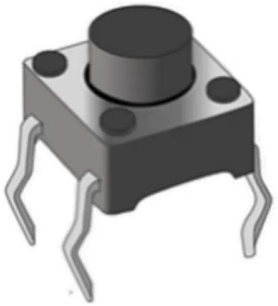
```
from gpiozero import LED
from time import sleep

# GPIO17 (BCM numbering)
led = LED(17)

# Accende il LED
led.on()
print("LED acceso")
sleep(2)

# Spegne il LED
led.off()
print("LED spento")
```

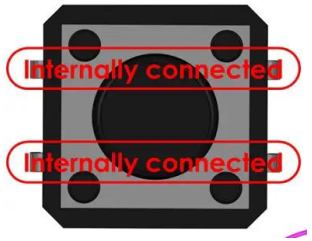


# LED blinking when button pressed

```
from gpiozero import LED, Button
from time import sleep

led = LED(17)
button = Button(2)

while True:
    button.wait_for_press()
    led.toggle()
    sleep(0.5)
```







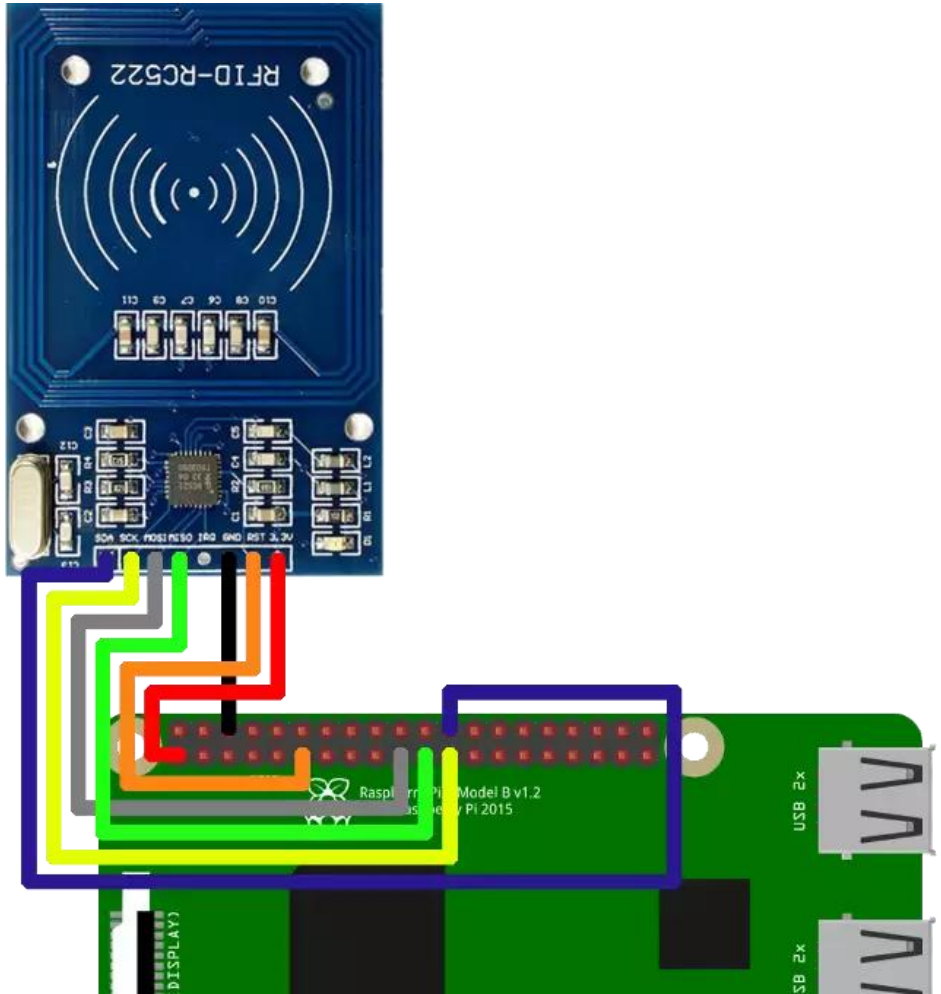
# Complichiamo l'esercizio… files + GPIO

Volete automatizzare la porta di accesso di camera vostra, in particolare, volete implementare:

- Un controllo accessi tramite chiave NFC
- Salvare un log degli eventi (ingresso/uscita), incluso l'orario

Extra (richiedono librerie aggiuntive!):

- Potete inserire un «notificatore» degli eventi tramite Telegram
- Inviare una notifica se un utente non ha timbrato il badge (usando anche un sensore PIR)
- Introdurre un meccanismo di apertura della porta automatizzato (servomotore aggangiato alla cerniera della porta, serratura elettromagetica, …)



# Libreria di Computer Vision: cv2

```
import cv2

# 1. Inizializza la cattura video (0 è solitamente l'indice della camera integrata)
cap = cv2.VideoCapture(0)

if not cap.isOpened():
  print("Errore: Impossibile accedere alla fotocamera")
  exit()

print("Premere 'q' per uscire.")

while True:
  # 2. Cattura il frame fotogramma per fotogramma
  ret, frame = cap.read()

  # Se il frame è stato catturato correttamente, ret è True
  if not ret:
    print("Errore nella ricezione del frame. Uscita...")
    break

  # 3. Visualizza il risultato in una finestra chiamata 'Webcam'
  cv2.imshow('Webcam', frame)

  # 4. Interrompe il ciclo se viene premuto il tasto 'q'
  # waitKey(1) attende 1 millisecondo tra un frame e l'altro
  if cv2.waitKey(1) & 0xFF == ord('q'):
    break

# 5. Rilascia la risorsa e chiude le finestre
cap.release()
cv2.destroyAllWindows()
```

OpenCV

# Domande, Dubbi, Perplessità?